

Understanding CNN Training Strategies for Image Classification

Intermediate Presentation – Explainable AI Project

Agenda / Outline

Part I:

- Motivation & project background
- CNN training intuition

Part II:

- Model design & comparison (A–D)

Part III:

- Result & Observation
- Conclusion & Outlook

Part I — Project Motivation & Scope

- Why understanding CNN training matters beyond accuracy
- Comparing **training strategies** instead of architectures
- Establishing strong baselines for later explainability analysis

Part I — CNN Training Intuition

- What happens during CNN training?
- Why training feels confusing at first
- How forward, backward, and optimization work together

Part I — CNN Training Intuition

From Input Images to Predictions

- Input images are processed in batches
- The model outputs **logits**, not probabilities
- A forward pass produces intermediate representations

`outputs = model(inputs)`

Part I — CNN Training Intuition

What Happens in a Forward Pass?

- Convolution layers extract hierarchical features
- Features become increasingly abstract
- The final layer maps features to class scores

Part I — CNN Training Intuition

Why Do We Need a Loss Function?

- Predictions alone are not enough for learning
- The loss measures **how wrong** the prediction is
- Cross-entropy connects logits with true labels

loss = criterion(outputs, labels)

Part I — CNN Training Intuition

Backward Pass: Learning from Errors

- Gradients show how parameters affect the loss
- Errors are propagated backward through the network
- Each layer receives an update signal

`loss.backward()`

Part I — CNN Training Intuition

Optimizer: Actual Updating the Model

- Optimizers use gradients to update weights
- Learning rate controls update size
- This step happens **after** backward pass

`optimizer.step()`

Part I — CNN Training Intuition

Putting Everything Together

- Forward pass: compute predictions
- Loss: measure prediction error
- Backward pass: compute gradients
- Optimizer: update parameters

```
outputs = model(inputs)
```

```
loss = criterion(outputs, labels)
```

```
loss.backward()
```

```
optimizer.step()
```

Part II — Model Design & Comparison

Why Do We Train Four Models?

- Accuracy alone does not explain model behavior
- Training strategy strongly influences learned representations
- We systematically vary **what is kept fixed** and **what is adapted**

Part II — Model Design & Comparison

Two Key Design Dimensions

- **Pretraining source**
 - ImageNet-pretrained
 - Flower-pretrained
- **Training strategy**
 - Linear probing (frozen backbone)
 - Fine-tuning (trainable backbone)

Part II — Model Design & Comparison

Overview of Models A–D

	Frozen backbone (linear probe)	Trainable backbone (fine-tune)
ImageNet pretrained	Model A	Model B
Flower pretrained (checkpoint)	Model C	Model D

Part II — Model Design & Comparison

Linear Probing vs Fine-Tuning

- Linear probing keeps feature extractor fixed
- Only the classification head is trained
- Fine-tuning allows features to adapt to the new task

Part II — Model Design & Comparison

Why Different Pretraining Sources Matter

- ImageNet provides generic visual features
- Flower-pretraining emphasizes domain-specific patterns
- Pretraining biases what the model focuses on

Part II — Model Design & Comparison

What We Expect to Learn from the Comparison

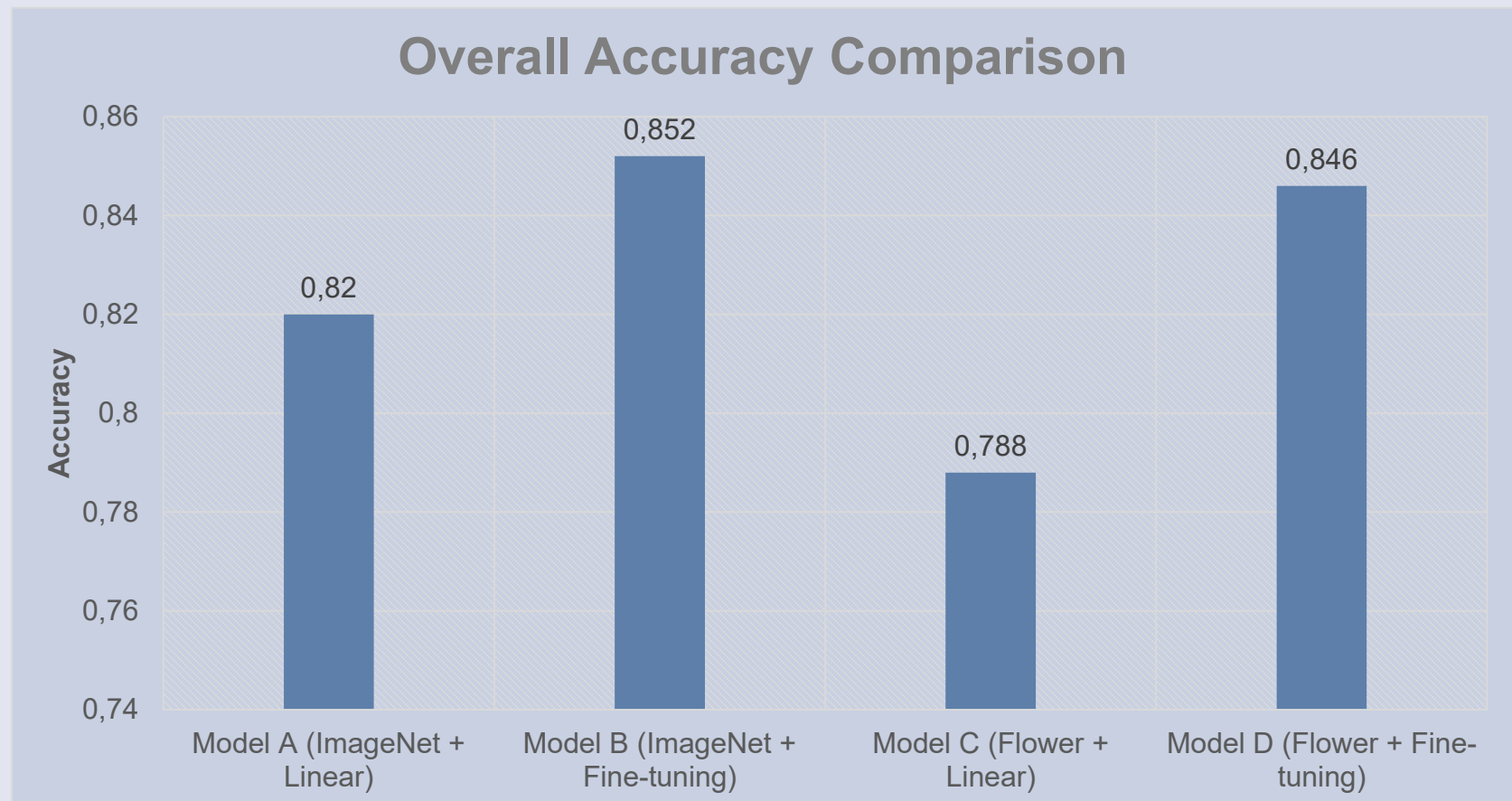
- When is fine-tuning worth the extra complexity?
- How transferable are pretrained features?
- How sensitive are models to domain shifts?

Part III — Results & Observations

How to Read the Results

- Accuracy summarizes performance, but hides behavior
- Loss reflects learning dynamics during training
- We focus on **relative differences**, not absolute numbers

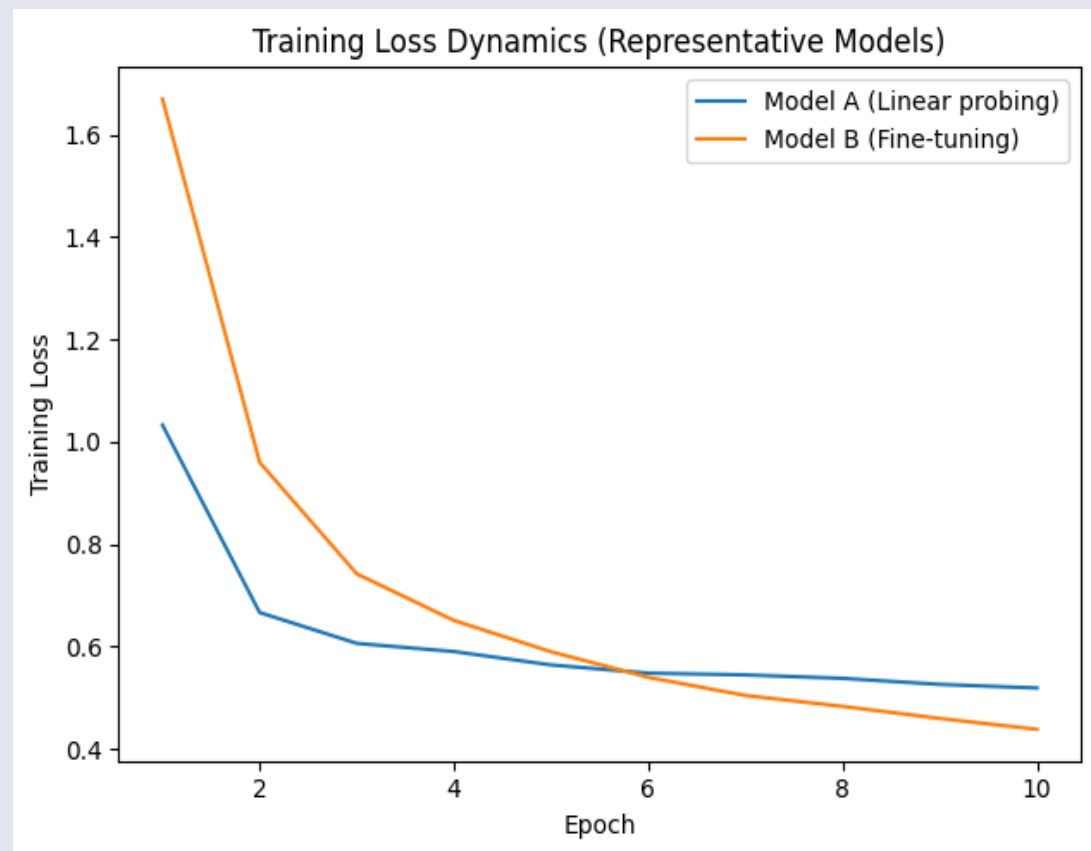
Part III — Results



Part III — Results & Observations

Training Loss Dynamics

- Fine-tuning shows a steeper loss reduction
- Linear probing converges earlier during training
- Loss curves highlight optimization dynamics



Part III — Results & Observations

Observed Patterns Across Models

- Fine-tuning consistently leads to higher performance than linear probing
- Linear probing converges earlier but saturates at a lower level
- ImageNet-pretrained models show more stable behavior across training strategies
- Flower-pretraining improves results only under specific conditions
- Similar accuracy values can correspond to different learning dynamics

Part III — Conclusion & Outlook

Key Takeaways

- CNN training involves multiple interacting components
- Training strategy matters as much as pretraining source
- Fine-tuning trades simplicity for adaptability
- Controlled comparisons are crucial for understanding model behavior

Part III — Conclusion & Outlook

Why These Results Matter

- Results provide a reliable baseline for explainability analysis
- Understanding training behavior is essential before interpreting models
- Metrics alone are insufficient for explaining model decisions

Part III — Conclusion & Outlook

Explainability-Oriented Analysis

- Apply explainability methods to trained models
- Compare explanations across training strategies
- Understand how training decisions shape learned representations